Lecture 25: File Transfer Protocol, FTP (RFC 959)

Objectives:

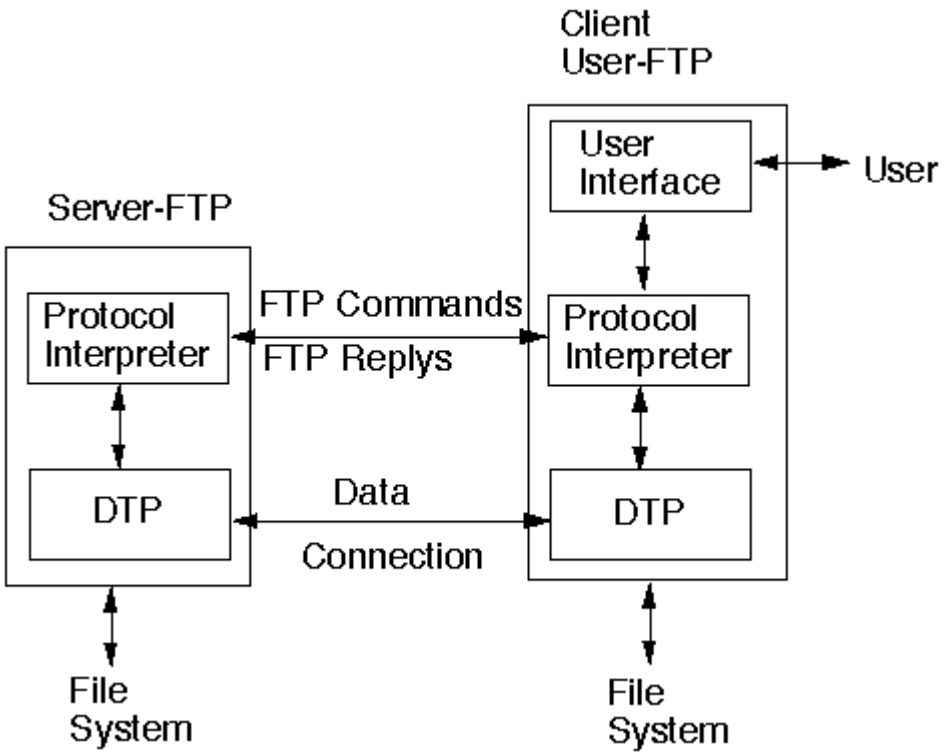
- Learn about the File Transfer Protocol (FTP)
  - Its Objectives and Architecture
  - Its functions (commands)
  - Its reply codes
- View a sample FTP session

1. Objectives and Architecture of FTP

The following were the design goals of the FTP protocol as stated in RFC 959:

- To promote sharing of files (computer programs and/or data)
- To encourage indirect or implicit (via programs) use of remote computers.
- To shield a user from variations in file storage systems among hosts.
- To transfer data reliably and efficiently.

The following figure shows the design architecture of the protocol.



The architecture suggests five software modules as described below:

User Interface	This provides a user interface and drives the client protocol interpreter.
Client PI	This is the client protocol interpreter. It issues commands to the remote server protocol interpreter and it also drives the client data transfer process.
Server PI	This is the server protocol interpreter which responds to commands issued by the client protocol interpreter and drives the server data transfer process.
Client DTP	This is the client data transfer process responsible for communicating with the server data transfer process and the local file system.
Server DTP	This is the server data transfer process responsible for communicating with the client data transfer process and the remote file system.

RFC 959 defines the commands by which the two PIs talk to each other and by which the two DTPs talk to each other.

The user interface and the mechanism by which the PIs talk to the DTPs are not part of the standard.

It is common practice for the PI and DTP functionalities to be part of the same program but this is not necessary.

There are two connections between the server and the client, both of which use TCP and function as follows:

Control connection:

This is used to communicate FTP commands and replies between the client PI and server PI. It is initiated by the client-PI.

FTP server listen for control connection on a well known port 21.

Control connection remains open as long as the client is connected and is not idle. It can be closed by the server if it remains idle for some time (usually some minutes).

Data Connection:

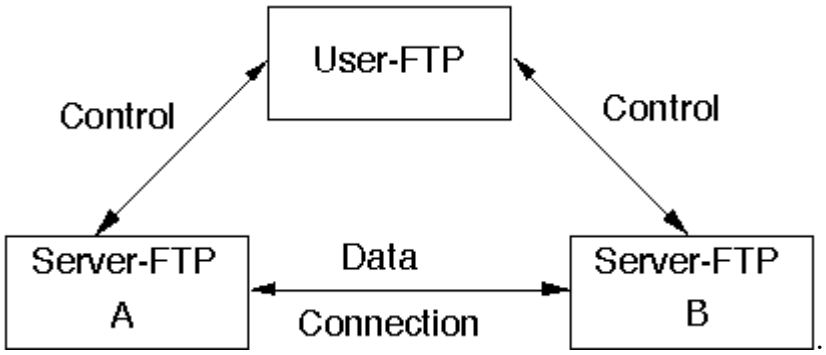
This is used to transfer data between the server and the client in response to some commands (store, retrieve, append, list, etc).

The connection is not permanent. It is opened only to service a command and it is closed immediately after the data is transferred for that command.

The protocol does not specify a port number for the data connection, but it provides two commands (PORT and PASV) that can be used to negotiate a port number – see the discussion on these commands below.

Also, the data port needs not be on the same host that initiates the control connection.

This means that a user can transfer data between two different servers it has control connections with, as shown by the following figure:



2. FTP Commands

FTP Commands are case insensitive and they consist of three to four ASCII characters followed by zero or more parameters.

They can be grouped into three categories:

- Access Control Commands
- Transfer Parameter Commands
- Service Commands

2.1 Access Control Commands

These are commands used to connect to the server and set the working directory.

NOTE: Commands preceded by * are generally not needed in modern computer systems.

USER *username*	Sends user identification to the server. This will normally be the first command transmitted by the user after establishing a control connection.
PASS *password*	This command must immediately follow the **USER** command, and, for most sites, completes the user's identification for access control. It is the responsibility of the user-Interface process to hide the password information.
*ACCT *account*	Some old sites require account information in addition to username and password. Most current systems do not require this.
CWD *path*	Changes the current working directory to the one specified by *path*.
CDUP	Changes directory to the parent of the current working directory
*SMNT *volume*	Loads another data storage structure
REIN	Terminates a USER, flushing all identification and I/O information, except to allow any data transfer in progress to be completed. All parameters are reset to the default settings and the control connection is left open.
QUIT	Terminates a USER and if data transfer is not in progress, the server closes the control connection. If data transfer is in progress, the connection will remain open for result response and the server will then close it.

2.2 Transfer Parameter Commands

Most data transfer parameters have default values, and the commands specifying data transfer parameters are required only if the default parameter values are to be changed.

If a transfer parameter is specified, it remains active until another one is specified.

Thus, the server must "remember" the applicable values.

The commands may be in any order except that they must precede the FTP service request commands discussed next.

PORT *h1,h2,h3,h4,p1,p2*	This is used to specify the IP address and port number to which the server should send data. h1,h2,h3,h4 represents the IP address in decimal, while p1, p2 represents a 16-bit port number. p1 is a decimal representing the left 8 bits and p2 represents the right 8 bits. Note: for a port number, n, p1 = n/256 p2 = n%256
PASV	This request the server to enter into a passive mode (listen mode) for data connection. The server will respond by sending the IP address and the port number on which it will be listening. The format of the response is: 227 Entering Passive mode (h1,h2,h3,h4,p1,p2) Note: From the above, the port number, n, can be computed as: n = p1*256 + p2
TYPE *A|E|I/L*	The two communicating systems may be representing data differently. Thus, data may need to be converted into a common form before transmission and then converted into local form at the other end. The options are: • A for ASCII (usually for text) • E for ABCDIC • I for Image (usually for binary data) • L for Local. This must be followed by the number of bytes. **Note:** Nowadays, only A and I are used.

*STRU *F|R|P*	This is used to allow data to be transferred reliably. It specifies the structure in which the file should be sent. The options are: • F for File: contiguous sequence of bytes – default • R for Record: sequence of records • P for Page: independent indexed pages **Note:** Nowadays, only File structure is used.
*MODE *S|B|C*	This is used to specify the transmission mode. The options are: • S for Streams: contiguous sequence of bytes – default • B for Block: each block is preceded by a header which contains the block number and size – this was used for error correction • C for Compressed: uses run-length compression algorithm. **Note:** Nowadays, only Stream mode is used.

2.3 Service Commands

The following commands define the file transfer functions that may be requested by a client.

The parameter is usually the file or directory and the result is normally sent through the data connection.

RETR *file*	Causes the server-DTP to transfer a copy of the file specified to the client-DTP at the other end of the data connection.
STOR *file*	Causes the server-DTP to accept the data transferred via the data connection and to store it as a file at the server site. If the file specified exists, its content is overridden.
*STOU *file*	Store unique - same as STOR except that it does not override an existing file.

APPE *file*	Causes the server-DTP to accept the data and to store it in a file at the server site. If the file specified exists, then the data shall be appended to that file.
ABOR	Abort the previous FTP service command and any associated transfer of data.
RNFR *old_file*	Specifies a filename that is to be renamed. RNTO should follow this command.
RNTO *new_file*	Renames the file specified in RNFR.
DELE *file*	Deletes the specified file from the server.
RMD *directory*	Removes the specified directory from the server.
MKD *directory*	Creates the specified directory on the server.
PWD	Returns the current working directory of the user.
LIST [*directory*]	Returns the directory listing of the specified directory. If no directory is specified, the current working directory is listed. The format of the directory listing is not specified in the protocol. Information such as file attributes, permissions and file size is sent.
NLST [*directory*]	Similar to LIST except only the list of the files and directories in returned – no attributes, etc.
SITE	Returns other services that the server provides. By using HELP SITE the user can get the list of available arguments for this command. No arguments are listed by the protocol but some common functions include "QUOTA" to access user quota information and "CHMOD" to change file permissions.
*STAT	Provides information about data transfer.
HELP [command]	Returns a list of the supported commands by the server. The optional argument can be used to provide more information about a particular command
SYST	Returns the operating system type on the server.
NOOP	Does nothing

3. FTP Replies

Each FTP command is responded with a reply code consisting of 3 digits ASCII numbers, followed by an optional message, followed by CRLF.

A reply message may also spam more than one line. In such a case, dash is appended after the three digits on each message line except the last.

The reply codes have the following general meanings:

Type	Description
1yz	Positive preliminary reply. Expect another reply before sending another command.
2yz	Positive completion reply. The last command completed successfully.
3yz	Positive intermediate reply. A further command must be sent.
4yz	Transient negative completion reply. The requested action did not take place but can be retried.
5yz	Permanent negative completion reply. The requested action did not take place and should not be retried.

The "y" digit encodes further information

Digit	Meaning
0	Syntax error
1	Information
2	Connection status.
3	Authentication and accounting.
4	Unspecified
5	File system status

Here are some typical reply codes.

125	Data connection already opened; transfer starting.
150	File status okay; about to open data connection.
200	Command okay.
202	Command not implemented, superfluous at this site.
211	System status.
212	Directory status.
213	File status.

214	Help message.
220	Service ready for new user.
221	Service closing control connection.
225	Data connection open; no transfer in progress.
226	Closing data connection.
227	Entering Passive Mode (h1,h2,h3,h4,p1,p2).
230	User logged in, proceed.
250	Requested file action okay, completed.
257	"PATHNAME" created.
331	User name okay, need password.
332	Need account for login.
350	Requested file action pending further information.
421	Service not available, closing control connection.
425	Can't open data connection.
426	Connection closed; transfer aborted.
450	Requested file action not taken.
451	Requested action aborted: local error in processing.
452	Requested action not taken.
500	Syntax error, command unrecognized.
501	Syntax error in parameters or arguments.
502	Command not implemented.
503	Bad sequence of commands.
504	Command not implemented for that parameter.
530	Not logged in.
532	Need account for storing files.
550	Requested action not taken.
551	Requested action aborted: page type unknown.
552	Requested file action aborted.
553	Requested action not taken.

4. Sample FTP Session

You can try interacting with FTP directly using TELNET. However, you need to direct all data from the server to another application/or socket running on a specific end-point.

You will specify the end-point using PORT command as discussed above. The port command should be followed by the particular data command.

```
Telnet mail.kfupm.edu.sa 21
220 itc6h1 FTP server (Version 4.1 Tue Aug 19 11:38:47 CDT 2003)
ready.
user gmbashir
331 Password required for gmbashir.
pass actual_password
230-Last login: Mon May 10 11:34:45 SAUST 2004 on /dev/pts/19 from
ics-bmghandi.pc.ccse.kfupm.edu.sa
230 User gmbashir logged in.
syst
215 UNIX Type: L8 Version: BSD-44
port 196,1,65,143,10,0
200 PORT command successful.
list
150 Opening data connection for /bin/ls.
226 Transfer complete.
quit
221 Goodbye.


Connection to host lost.
```

The following is the result of the list command captured by the **ReadReceiver** application running on the above port:

